

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**
Aula: 09 - Mais algoritmos de otimização

Professor: **Raphael Martins Brum**

2026

(Aula passada) Introdução à otimização

- **Objetivo:** encontrar a melhor solução para um problema
- **Problemas** otimizáveis e suas **soluções:**
 - Rota de menor custo: endereço 1 -> endereço 2 -> endereço 3 -> ... -> endereço 5
 - Máximo lucro em carteira de investimentos: 20% no Ativo 1, 80% no Ativo 2
 - Alocação de tarefas: Tarefa 1 no PC 4, Tarefa 2 no PC 2, Tarefa 3 no PC 5
- Otimizar $\Leftrightarrow$ Minimizar ou maximizar alguma métrica



(Aula passada) Introdução à otimização

- Espaço de busca: conjunto de todas as possíveis soluções para um problema (contínuo x discreto)



Programação Evolutiva

- Metaheurística
- Classe de algoritmos de otimização inspirados na Teoria da Evolução
- São exemplos desses algoritmos:
 - Algoritmo Genéticos e suas variantes (cruzamentos dominam)
 - Estratégias evolutivas (mutação domina)
 - Programação Genética e suas variantes
 - ...
- Aplicáveis mesmo em situações em que os algoritmos de otimização clássicos fracassam
 - Funções objetivo não-contínuas, ruidosas
 - Funções objetivo “caixa preta”



Conceitos de Evolução

Definições:

- **Indivíduo** = solução candidata
- **Gene** = parâmetro da solução
- **Alelo** = valor de um gene
- **População** = conjunto de soluções
- **Aptidão (fitness)** = qualidade da solução
- **Geração** = população resultante após uma iteração do algoritmo

Processos de Evolução / Operadores genéticos

- Seleção
- *Crossover*
- Mutação
- ...



Fluxo do Algoritmo Genético

```
# pseudocódigo
populacao = inicializar(tamanho=100)
for geracao in range(MAX_GERACOES):
    fitness = [avaliar(ind) for ind in populacao]
    pais = selecionar(populacao, fitness)
    filhos = crossover(pais)
    filhos = mutar(filhos, taxa=0.01)
    populacao = filhos
melhor = max(populacao, key=avaliar)
```



AG: Um primeiro exemplo: $\max. f(x) = x^2$, x entre 0 e 31

```
import random

# Cada indivíduo é um número de 5 bits, ex: [1,0,1,1,0] = 22
def bits_para_int(bits):
    return int("".join(str(b) for b in bits), 2)

def fitness(bits):
    x = bits_para_int(bits)
    return x ** 2

def crossover(pai, mae):
    ponto = random.randint(1, 4)
    filho1 = pai[:ponto] + mae[ponto:]
    filho2 = mae[:ponto] + pai[ponto:]
    return filho1, filho2
```



AG: Um primeiro exemplo: maximizar $f(x) = x^2$

```
def mutar(ind, taxa=0.1):
    return [1-b if random.random() < taxa else b for b in ind]

pop = [[random.randint(0, 1) for _ in range(5)] for _ in range(6)]

for geracao in range(10):
    pop.sort(key=lambda ind: fitness(ind), reverse=True)
    elite = pop[:2]
    f1, f2 = crossover(pop[0], pop[1])
    f3, f4 = crossover(pop[0], pop[2])
    pop = elite + [mutar(f) for f in [f1, f2, f3, f4]]

melhor = max(pop, key=fitness)
print(f"Melhor: x={bits_para_int(melhor)}, f(x)={fitness(melhor)}")
```



Operadores de Seleção

```
def selecao_roleta(pop, fits):  
    total = sum(fits)  
    probs = [f/total for f in fits]  
    idx = random.choices(  
        range(len(pop)), weights=probs, k=2  
    )  
    return pop[idx[0]], pop[idx[1]]
```

```
def selecao_torneio(pop, fits, k=3):  
    candidatos = random.sample(  
        list(zip(pop, fits)), k  
    )  
    return max(candidatos,  
        key=lambda x: x[1])[0]
```

Método	Pressão seletiva	Vantagem	Desvantagem
Roleta	Alta quando fits variam muito	Proporcional ao fitness	Dominância de superindivíduos
Torneio	Controlada pelo k	Simples, eficiente	Ignora magnitude do fitness
Truncamento	Muito alta	Converge rápido	Perde diversidade
Ranking	Uniforme	Evita dominância	Mais lento



Operadores de *crossover*

```
# 1 ponto
def cx_1ponto(p1, p2):
    c = random.randint(1, len(p1)-1)
    return p1[:c]+p2[c:], p2[:c]+p1[c:]

# 2 pontos
def cx_2pontos(p1, p2):
    a, b = sorted(random.sample(range(len(p1)), 2))
    return p1[:a]+p2[a:b]+p1[b:], \
           p2[:a]+p1[a:b]+p2[b:]
```

```
# Uniforme (cada gene: 50% chance de trocar)
def cx_uniforme(p1, p2):
    f1 = [a if random.random() < .5 else b
           for a, b in zip(p1, p2)]
    f2 = [b if random.random() < .5 else a
           for a, b in zip(p1, p2)]
    return f1, f2
```

- Mistura duas soluções para criar uma (ou mais) soluções “filhas”



Operadores de *mutação*

```
# Bit flip (binário)
def mut_bitflip(ind, taxa=0.01):
    return [1-g if random.random() < taxa
            else g for g in ind]

# Gaussiana (valores reais)
def mut_gaussiana(ind, sigma=0.1):
    return [g + random.gauss(0, sigma)
            for g in ind]
```

```
# Troca (permutação)
def mut_swap(ind):
    i, j = random.sample(range(len(ind)), 2)
    ind[i], ind[j] = ind[j], ind[i]
    return ind
```

- *Trade-off* da taxa de mutação (típica: 0,1% a 2%):
 - Muito alta = algoritmo tende a ser uma busca aleatória
 - Muito baixa = algoritmo converge para uma solução não-ótima



Exemplo do Problema da Mochila 0/1

```
import random

def criar_item(nome, peso, valor):
    return {"nome": nome, "peso": peso, "valor": valor}

itens = [
    criar_item("Notebook", peso=3.0, valor=300),
    criar_item("Câmera", peso=1.0, valor=200),
    criar_item("Livros", peso=4.0, valor=150),
    criar_item("Fone", peso=0.5, valor=80),
    criar_item("Tablet", peso=2.0, valor=240),
]

CAPACIDADE = 5.0
N = len(itens)
```



Exemplo do Problema da Mochila 0/1

```
def fitness(crom):  
    """Valor total dos itens selecionados; 0 se exceder a capacidade."""  
    peso = sum(itens[i]["peso"] * crom[i] for i in range(N))  
    valor = sum(itens[i]["valor"] * crom[i] for i in range(N))  
    return valor if peso <= CAPACIDADE else 0  
  
def torneio(pop, fits, k=3):  
    """Retorna o melhor entre k candidatos sorteados."""  
    candidatos = random.sample(list(zip(pop, fits)), k)  
    return max(candidatos, key=lambda x: x[1])[0]  
  
def crossover(p1, p2):  
    c = random.randint(1, N - 1)  
    return p1[:c] + p2[c:], p2[:c] + p1[c:]
```



Exemplo do Problema da Mochila 0/1

```
# mutação bit-flip
def mutar(crom, taxa=0.02):
    return [1 - g if random.random() < taxa else g for g in crom]

# algoritmo genético
def ag_mochila_01(pop_size=50, geracoes=200, taxa_cx=0.8, taxa_mut=0.02, elitismo=2):
    pop = [[random.randint(0, 1) for _ in range(N)] for _ in range(pop_size)]
    hist_melhor, hist_medio = [], []

    for _ in range(geracoes):
        fits = [fitness(ind) for ind in pop]
        hist_melhor.append(max(fits))
        hist_medio.append(sum(fits) / pop_size)
```



Exemplo do Problema da Mochila 0/1

```
# elitismo: preserva os melhores diretamente
ordem = sorted(range(pop_size), key=lambda i: fits[i], reverse=True)
nova_pop = [pop[i] for i in ordem[:elitismo]]

while len(nova_pop) < pop_size:
    p1 = torneio(pop, fits)
    p2 = torneio(pop, fits)
    if random.random() < taxa_cx:
        f1, f2 = crossover(p1, p2)
    else:
        f1, f2 = p1[:], p2[:]
    nova_pop += [mutar(f1, taxa_mut), mutar(f2, taxa_mut)]

pop = nova_pop[:pop_size]

melhor = max(pop, key=fitness)
return melhor, hist_melhor, hist_medio
```



Exemplo do Problema da Mochila 0/1

```
# execução
random.seed(7)
melhor, hist_melhor, hist_medio = ag_mochila_01()

print(f"Cromossomo: {melhor}")
print(f"Valor total: R$ {fitness(melhor):.0f}")
peso_total = sum(itens[i]["peso"] * melhor[i] for i in range(N))
print(f"Peso total: {peso_total:.1f} kg / {CAPACIDADE} kg")
```



Exemplo do Problema da Mochila (fracionária)

```
# gráfico de convergência
geracoes = range(1, len(hist_melhor) + 1)

fig, ax = plt.subplots(figsize=(9, 4))

ax.plot(geracoes, hist_melhor,
        color="#534AB7", linewidth=2, label="Melhor fitness")
ax.plot(geracoes, hist_medio,
        color="#5DCAA5", linewidth=1.5, label="Fitness médio", linestyle="--")

# linha de referência: valor ótimo conhecido
ax.axhline(520, color="#D85A30", linewidth=1, linestyle=":", label="Ótimo (R$ 520)")
```



Exemplo do Problema da Mochila (fracionária)

```
ax.set_xlabel("Geração")
ax.set_ylabel("Fitness (R$)")
ax.set_title("Convergência do AG - mochila 0/1")
ax.legend()
ax.set_xlim(1, len(hist_melhor))
ax.set_ylim(0, max(hist_melhor) * 1.15)
ax.grid(True, linewidth=0.4, alpha=0.5)

fig.tight_layout()
plt.show()
```



Exemplo do Problema da Mochila (fracionária)

- Não faz sentido usar AG, pois a solução gulosa é ótima!
- Mas vamos em frente...
- Precisamos de soluções fracionárias
 - Genes pertencerão ao intervalo $[0.0, 1.0]$
 - Precisamos de um *crossover* adaptado: vamos utilizar o BLX-alpha (*blended crossover*):
 - $\alpha * p1 + (1-\alpha) * p2$
 - A mutação não pode ser *bit flip*: vamos utilizar mutação gaussiana
 - Somamos a cada parâmetro um ruído distribuído em $N(0, \sigma)$.
 - Podemos gerar soluções inválidas
 - em vez de jogá-las fora, vamos corrigi-las
- Prole será composta de 80% crossover, 20% cópias com mutação
- Vamos ficar com as duas melhores soluções de cada prole (elitismo)



Exemplo do Problema da Mochila (fracionária)

```
import random

def criar_item(nome, peso, valor):
    return {"nome": nome, "peso": peso, "valor":
valor,
            "densidade": valor / peso}

itens = [
    criar_item("Notebook", 3.0, 300),
    criar_item("Câmera", 1.0, 200),
    criar_item("Livros", 4.0, 150),
    criar_item("Fone", 0.5, 80),
    criar_item("Tablet", 2.0, 240),
]
CAPACIDADE = 5.0
N = len(itens)
```

```
# reparação / legalizacao
def reparar(crom):
    peso = sum(itens[i]["peso"] * crom[i] for
i in range(N))
    if peso > CAPACIDADE:
        escala = CAPACIDADE / peso
        crom = [g * escala for g in crom]
    return crom
```



Exemplo do Problema da Mochila (fracionária)

```
# fitness
def fitness(crom):
    return sum(itens[i]["valor"] * crom[i] for i in
range(N))

# operadores genéticos
def crossover_aritmetico(p1, p2):
    #blended crossover
    alpha = random.random()
    filho = [alpha * a + (1 - alpha) * b
    for a, b in zip(p1, p2)]
    return reparar(filho)
```

```
def mutar(crom, taxa=0.2, sigma=0.15):
    novo = []
    for g in crom:
        if random.random() < taxa:
            g = max(0.0, min(1.0, g +
random.gauss(0, sigma)))
        novo.append(g)
    return reparar(novo)

def torneio(pop, fits, k=3):
    candidatos = random.sample(list(zip(pop,
fits)), k)
    return max(candidatos, key=lambda x:
x[1])[0]
```



Exemplo do Problema da Mochila (fracionária)

```
def ag_fracionario( pop_size=80, geracoes=300, taxa_cx=0.8, taxa_mut=0.2, elitismo=2):  
    # população inicial: frações aleatórias em [0, 1], depois reparadas  
    pop = [reparar([random.random() for _ in range(N)]) for _ in range(pop_size)]  
    historico = []  
  
    for _ in range(geracoes):  
        fits = [fitness(ind) for ind in pop]  
        historico.append(max(fits))  
  
        # elitismo: os melhores passam direto para a próxima geração  
        indices_elite = sorted(range(pop_size), key=lambda i: fits[i], reverse=True)  
        nova_pop = [pop[i] for i in indices_elite[:elitismo]]
```



Exemplo do Problema da Mochila (fracionária)

```
while len(nova_pop) < pop_size:
    p1 = torneio(pop, fits)
    p2 = torneio(pop, fits)
    filho = crossover_aritmetico(p1, p2) if random.random() < taxa_cx \
        else p1[:]
    nova_pop.append(mutar(filho, taxa_mut))

pop = nova_pop

melhor = max(pop, key=fitness)
return melhor, historico
```



Exemplo do Problema da Mochila (fracionária)

```
# execução e exibição
random.seed(42)
melhor, historico = ag_fracionario()

print(f"{'Item':<12} {'Fração':>8} {'Peso usado':>12} {'Valor':>10}")
print("-" * 46)
for i, frac in enumerate(melhor):
    item = itens[i]
    print(f"{'item['nome']':<12} {'frac':>8.1%}"
          f"{'item['peso']*frac':>10.2f}kg {'item['valor']*frac':>9.2f}")
print("-" * 46)
print(f"{'Valor total':>34} R${fitness(melhor):>8.2f}")
print(f"Greedy ótimo: R$ 670.00")
print(f"Gap: {(670 - fitness(melhor)) / 670:.4%}")
```



Exemplo do Problema da Mochila (fracionária)

```
# gráfico de convergência
geracoes = range(1, len(hist_melhor) + 1)

fig, ax = plt.subplots(figsize=(9, 4))

ax.plot(geracoes, hist_melhor,
        color="#534AB7", linewidth=2, label="Melhor fitness")
ax.plot(geracoes, hist_medio,
        color="#5DCAA5", linewidth=1.5, label="Fitness médio", linestyle="--")

# linha de referência: valor ótimo conhecido
ax.axhline(520, color="#D85A30", linewidth=1, linestyle=":", label="Ótimo (R$ 520)")

ax.set_xlabel("Geração")
ax.set_ylabel("Fitness (R$)")
ax.set_title("Convergência do AG - mochila 0/1")
ax.legend()
ax.set_xlim(1, len(hist_melhor))
ax.set_ylim(0, max(hist_melhor) * 1.15)
```



Projeto 9: TSP com diferentes algoritmos

Faça sua própria implementação dos algoritmos de otimização vistos aqui para resolver os **benchmarks** de TSP da TSPLIB95, encontráveis em <https://github.com/mastqe/tsplib>.

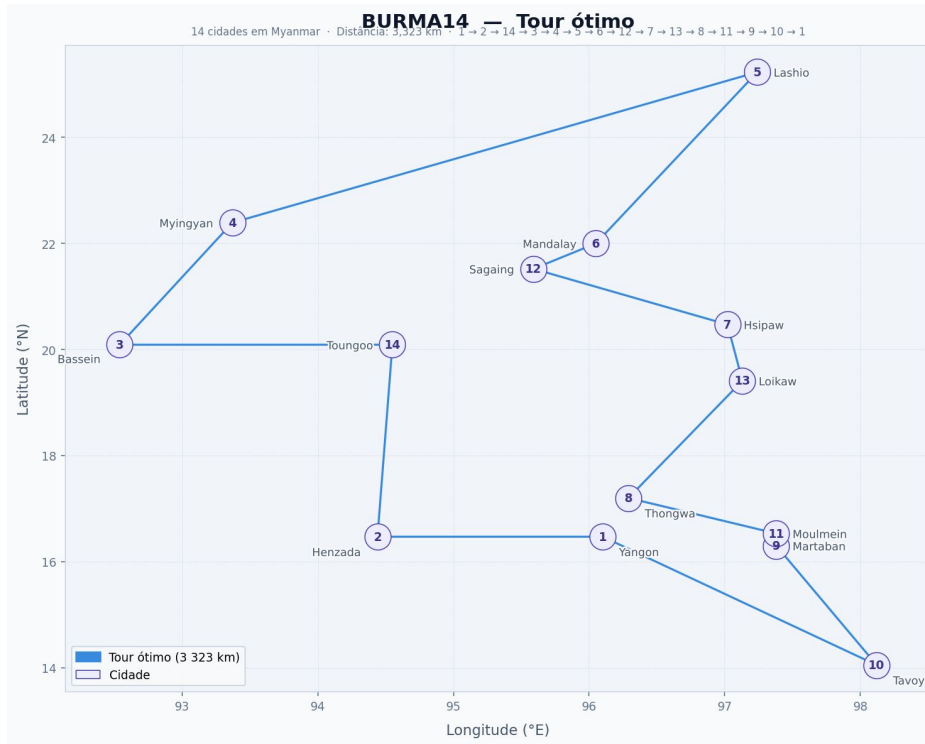
Registre os resultados para cada benchmark em uma tabela.

Utilize o código abaixo para fazer a leitura dos benchmarks.

```
# pip install tsplib95
# PARSER
import tsplib95
prob = tsplib95.load('berlin52.tsp')
nos = list(prob.get_nodes())
```



TSPLIB - Problema “burma14”



NAME: burma14
TYPE: TSP
COMMENT: 14-Staedte in Burma (Zaw Win)
DIMENSION: 14
EDGE_WEIGHT_TYPE: GEO
EDGE_WEIGHT_FORMAT: FUNCTION
DISPLAY_DATA_TYPE: COORD_DISPLAY
NODE_COORD_SECTION

1	16.47	96.10
2	16.47	94.44
3	20.09	92.54
4	22.39	93.37
5	25.23	97.24
6	22.00	96.05
7	20.47	97.02
8	17.20	96.29
9	16.30	97.38
10	14.05	98.12
11	16.53	97.38
12	21.52	95.59
13	19.41	97.13
14	20.09	94.55

EOF

